

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

DevMerge

**AN INTELLIGENT COLLABORATIVE LEARNING, PRACTICE AND ACADEMIC
MANAGEMENT PLATFORM**

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Navya Aggarwal (2210991967)

Supervised By:

**Dr. Shikha Tuteja
Associate professor
Chitkara University**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
	Declaration	3
	Acknowledgement	4
	Abstract	5
1	Introduction	6-9
2	Methodology	10-14
3	Tools and Technologies	15-16
4	Implementation	17-18
5	Major Findings/Outcomes/Output/Results	19
6	Future Scope	20
7	Conclusion	21
8	References	22
9	Appendices	23-27

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**DevMerge–An Intelligent Collaborative Learning, Practice and Academic Management Platform**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Shikha Tuteja**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Navya Aggarwal (2210991967)

Date: **28-04-2026**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja

Associate professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We express our sincere gratitude to our supervisor **Dr. Shikha Tuteja**, Associate Professor, Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for her continuous guidance, valuable suggestions, and constant encouragement throughout the development of this project.

We would also like to thank the faculty members of the Department of Computer Science and Engineering for providing us with the necessary resources and academic support required to complete this project successfully.

We are equally grateful to our friends and family members for their motivation and support during the course of this work.

Finally, we extend our appreciation to all those who directly or indirectly contributed to the successful completion of this project.

Navya Aggarwal

2210991967

ABSTRACT

The project titled **“DevMerge – An Intelligent Collaborative Learning, Practice and Academic Management Platform”** aims to design and develop a comprehensive web-based application that streamlines and automates various operations within an educational institution. This system provides a centralized platform for managing academic resources, assignments, grading workflows, and collaborative technical development efficiently. The primary objective of the project is to eliminate fragmented workflows, reduce the administrative burden on educators, and improve the learning trajectory of students through digital transformation and tailored feedback systems.

The application incorporates secure authentication mechanisms, role-based access control, and an intuitive, responsive interface to ensure efficient interaction for students, faculty, and institutional administrators alike. It also enables advanced features such as an integrated, LeetCode-like coding practice environment, a real-time collaborative playground for team development, and an AI-driven performance engine that evaluates code metrics—such as accuracy and time complexity—to generate personalized readiness indexes.

The system has been developed using a modern, scalable full-stack architecture featuring React.js on the frontend, Node.js and Express.js for the backend services, and MongoDB for secure, real-time data management. The implementation demonstrates how integrated software solutions and intelligent analytics can significantly enhance operational efficiency and pedagogical outcomes in the healthcare domain of academic ecosystems.

1. Introduction

1.1. Overview of the Project

DevMerge is a web-based application designed to modernize and streamline the day-to-day operations of educational institutions. In many schools and colleges today, administrative and pedagogical tasks—such as resource distribution, assignment management, testing, and student evaluation—are still handled through fragmented systems that do not work well together. This often leads to inefficiencies, increased workload, and a higher probability of academic disconnect for both students and educators. The proposed system aims to provide a comprehensive digital solution that integrates all essential academic and technical development operations into a single unified platform. By leveraging modern full-stack web technologies, the application enables seamless interaction between students, faculty members, and institutional administrators. It ensures that all information—ranging from structured course notes to complex programming submissions—is stored in a structured and easily accessible format, reducing redundancy and improving data consistency.

The system centralizes key functionalities such as institutional academic management, centralized content distribution, LeetCode-style coding practice, and automated evaluation tools. This centralization not only improves coordination among stakeholders but also enhances decision-making through quick access to accurate performance metrics via a dedicated teacher analytics dashboard. Additionally, the platform is designed with a user-friendly, responsive interface and features a real-time collaborative playground, ensuring deep engagement and ease of use for individuals working on collective programming tasks. Overall, DevMerge contributes to the digital transformation of educational services by improving operational efficiency, minimizing administrative burdens, and enhancing the quality of technical learning through personalized, AI-driven feedback.

1.2. Problem Statement

Despite advancements in technology, many educational institutions continue to rely on traditional or partially digital systems for managing their academic and technical operations. These systems often suffer from several limitations, including a lack of cross-platform integration, inefficient performance tracking, and limited accessibility. Manual academic record-keeping or handling programming assignments through disparate text files can lead to missing submissions, duplication of data, and difficulty in retrieving a student's technical history when required.

Semi-digital workflows—while an improvement over legacy methods—still lack real-time synchronization, interactive testing, and centralized management. This creates inconsistencies and delays in accessing critical assessment metrics, which can negatively impact an educator's pedagogical insights and a student's skill trajectory. Furthermore, standard platforms lack dedicated, localized testing environments for modern development skills.

Another major concern is the lack of secure, role-based access control in many existing academic systems. Sensitive student performance data, exam metrics, and administrative control paths must be protected from unauthorized access. The absence of proper multi-tier authentication mechanisms increases the risk of data breaches and internal privilege elevation.

Therefore, there is a strong need for a scalable, secure, and intelligent academic ecosystem like DevMerge that can address these challenges. The system should provide real-time collaborative development modules, ensure comprehensive code execution monitoring, and offer a seamless, responsive user experience for students, faculty, and institutional administrators alike.

1.3. Objectives of the Project

The primary objective of DevMerge is to develop an efficient, reliable, and intelligent learning management and practice platform that simplifies and automates various administrative and operational processes. The system is designed with the following specific objectives:

- To create a centralized platform for managing all institution-related learning activities, including course notes, role-based dashboards, and student analytics.
- To minimize manual resource overhead and reduce human error in processing technical assignments and code assessments.
- To implement secure authentication mechanisms using modern technologies such as JWT to ensure student data privacy and system protection.
- To enable efficient technical challenge testing and evaluation, minimizing evaluation wait times and improving overall laboratory resource utilization.
- To provide advanced progress analytics and filtering capabilities, allowing faculty members to map individual student skill graphs and pinpoint structural weak spots.
- To design an immersive, user-friendly, and responsive interface that enhances accessibility and interaction across different screen sizes using Tailwind CSS.
- To ensure high structural scalability of the system so that it can be seamlessly extended with additional collaborative, administrative, or predictive features in the future.

These objectives collectively aim to improve the overall operational flow of institutions while radically enhancing the learning and practice experience of both students and educators.

1.4. Scope of the Project

The scope of DevMerge includes the design and development of a fully functional collaborative learning and coding platform that addresses the core requirements of modern technical education. The system primarily focuses on managing course distribution, hosting integrated development sandboxes, and facilitating communication between instructors and learners.

The application allows students to register, securely authenticate, browse institutional tracks, practice programming in an embedded platform, and monitor their learning velocity. Faculty members can manage specialized curricula, view live analytics graphs, and dynamically deploy technical challenges. Administrators, Heads of Departments (HODs), and Super Admins can oversee cross-departmental configurations, system analytics, and security baselines.

While the current deployment covers essential learning systems, it is engineered with maximum scalability in mind. Future enhancements can include advanced predictive analytics for automated course personalization, deeper integration with decentralized cloud compilers, automatic plagiarism and code duplication filters, and immersive virtual classroom tools.

The project is delivered as an optimized, responsive web-based platform in its current implementation; however, its modular nature allows it to be extended to mobile frameworks to enhance daily usage and engagement. In summary, the scope of this project is to provide a robust foundation for a modern academic ecosystem that naturally evolves with technological advancements and growing educational demands.

2. Methodology

2.1. Development Approach

The development of DevMerge follows the Agile software development methodology, which emphasizes iterative progress, flexibility, and continuous improvement. Unlike rigid, traditional models such as the Waterfall approach, Agile allows the project to evolve through multiple cycles, enabling the development team to adapt to changing architectural needs and incorporate continuous stakeholder feedback at every deployment stage.

The project was divided into smaller, self-contained functional modules, and each module was iteratively coded, tested, and optimized. Regular review sessions helped in identifying integration friction early and improving user experience incrementally. This approach ensured superior code maintainability, faster resolution of cross-platform anomalies, and a highly polished final product.

2.2. Requirement Analysis

Requirement analysis is a crucial phase in the development process, where the needs and expectations of end users are carefully documented. In this project, requirements were identified by analyzing the typical workflow of educational institutions and technical laboratories, mapping the interactions between students, faculty members, and structural administrators.

The project requirements were categorized into two main pillars:

2.2.1. Functional Requirements

These define the core execution paths and services of the application:

- Multi-portal user registration and token-based authentication (Student, Faculty, Admin, HOD, and Super Admin).
- Dynamic course management, material distribution, and assignment submission channels.
- An integrated code testing environment supporting runtime validation and complexity tracking.
- Centralized real-time analytics pipelines and institutional tracking boards.
- A responsive interface capable of shifting seamlessly between standard layouts and immersive code environments.

Non-Functional Requirements

These focus on performance vectors and structural stability:

- High cryptographic data protection and persistent session guardrails.
- Architectural scalability to support large batches of concurrent compilation and analytical requests.
- Low-latency UI updates using lightweight client-side state hooks.
- Fault-tolerant API exception handling and high system availability.

This comprehensive phase ensured that the platform was tailored to meet real-world academic challenges with high performance.

2.3. System Design

The system design phase involved detailing the technical architecture and underlying schema configurations to achieve optimal efficiency, scalability, and long-term code maintainability.

2.3.1. Architecture Design

The application is structured upon a highly decoupled Client-Server Architecture:

- **The Frontend (Client-Side):** Developed as an expressive Single Page Application (SPA) to manage interface navigation and localized responsive rendering.
- **The Backend (Server-Side):** Implemented as an independent, high-throughput REST API layer handling core business logic, session validations, and microservices orchestration.
- **The Database:** Handles the reliable retention, modification, and referencing of complex persistent schemas.

2.3.2. Database Design

A flexible, high-performance schema model was deployed using MongoDB. Document collections were structured to store and index user profiles, course data arrays, complex coding submissions, and structural analytics. Document relationships were established utilizing referenced models to maintain high data consistency, avoid hard redundancies, and support swift analytical lookups.

2.3.3. User Interface Design

The user interface was crafted to be modern, immersive, and highly responsive, ensuring comfortable navigation for users across varying technical proficiencies. Detailed wireframes and user flow hierarchies were mapped out prior to implementation, focusing on dark mode accessibility configurations and fluid, motion-driven interactive states.

2.4. Development Process

The development phase focused on transforming the architectural designs into clean, functional code utilizing a modern full-stack ecosystem.

2.4.1. Frontend Development

The user-facing client layer was constructed using **React 18**, exploiting its modern component-driven architecture, concurrent features, and custom hooks. Styling execution was handled via **Tailwind CSS**, allowing fast utility-first layout building and clean cross-device responsiveness. Interactive elements and layout shifts were enhanced using **Framer Motion** for a premium, accessible visual feel.

2.4.2. Backend Development

The application server was built on top of **Node.js** and **Express.js**, delivering a non-blocking, event-driven foundation for building secure RESTful endpoints. The API was structured modularly to separate controllers, route protection gates, and database model handlers, maximizing processing performance.

2.4.3. Database Integration

MongoDB served as the data foundation, accessed via custom abstraction methods and optimized data connectors (**Mongoose/Axios API bindings**). This enabled smooth, low-latency interaction for indexing, complex querying, and reading system logs without blocking user flows.

2.4.4. Authentication and Authorization

Secure state persistence was established utilizing **JSON Web Tokens (JWT)** alongside role-based access tokens. This stack governs the multi-role route guards (Student, Faculty, Admin, HOD, and Super Admin), completely neutralizing arbitrary parameter manipulation or unauthorized data visibility.

2.5. Testing and Deployment

Comprehensive testing paradigms were applied to validate the platform's stability, reliability, and security profiles under active workloads:

2.5.1. Testing

- **Unit Testing:** Individual UI components, utility logic hooks, and isolated controller functions were individually tested to ensure input-output accuracy.
- **Integration Testing:** End-to-end testing cycles were carried out to verify seamless communication across the React engine, the Express route controllers, and the MongoDB document layers.
- **User Testing:** Real-world educational workflows were replicated across target demographics to optimize visual hierarchies, verify role access boundaries, and eliminate workflow friction.

2.5.2. Deployment

Following successful validation phases, the production application was securely distributed to cloud hosting environments:

- The **React frontend layer** was optimized and deployed on **Vercel / Netlify**, guaranteeing low latency edge-network delivery, automatic scaling, and continuous deployment tracking.
- The **Node.js backend API and orchestration layers** were containerized and hosted on scalable cloud compute structures (such as **AWS S3/Docker instances**), utilizing environment variables to maintain optimal runtime configuration security.

3. Tools and Technologies

3.1. Frontend

- **React 18:** Leveraged for building component-driven interfaces, optimizing DOM updates, and handling concurrent states.
- **React Router 6:** Governs client-side multi-tier routing architectures and route guard controls.
- **Tailwind CSS:** Employed for styling, guaranteeing consistency through design variables and deep mobile-first responsiveness.
- **Framer Motion:** Used to drive semantic UI/UX transitions, state changes, and spatial micro-animations.
- **Zustand:** Deployed for lightweight, performant, and non-intrusive global client state management.

3.2. Backend

- **Node.js & Express.js:** Functions as the core backend engine, enabling fast, asynchronous request handling.
- **Axios:** Serves as the central HTTP client layer, using custom interceptors to pass credentials securely.

3.3. Database

- **MongoDB:** Relied upon as the primary non-relational database for storing unstructured and semi-structured academic models, challenge sets, and analytical data securely.

3.4. Security

- **JWT (JSON Web Tokens):** Handles token-based state authorization, session persistence, and secure token expiration routines.
- **Role-Based Access Control Guards:** Deeply integrated into both client navigation and backend route structures to enforce privilege isolation.

3.5. Additional Tools and Services

- **Heroicons:** Integrated for clean, optimized, SVG-driven icon consistency.
- **Prettier & ESLint:** Utilized within the engineering pipeline to enforce absolute code formatting standards and clean execution metrics.
- **Git & GitHub:** Utilized for complete decentralized version control, code tracking, and peer code analysis.

4. Implementation

4.1. Architecture

The DevMerge framework is built around a scalable client-server model designed to isolate presentation logic completely from systemic business layers.

The React-driven frontend application acts as the immediate interaction canvas, dispatching semantic JSON payloads over HTTPS. The Express-driven backend processes these incoming streams, handles permission verifications, performs any necessary computing logic, and writes back cleanly to the MongoDB cluster. This structural isolation ensures high modifiability, allowing developers to upgrade backend compilation engines or frontend visual sets independently.

4.2. Features

DevMerge incorporates a comprehensive set of advanced features designed to accelerate learning, automate academic administration, and ensure secure data tracking:

- **Multi-Portal Role Isolation:** Tailored dashboards for five specific user roles (Student, Faculty, Admin, HOD, and Super Admin), providing explicit view boundaries and operations tracking.
- **The Operational Hub:** A centralized workspace for students to track active coursework, deadlines, and live metrics.
- **The Algorithm Vault:** An embedded, LeetCode-style coding sandbox allowing students to execute code and receive structural feedback inside the ecosystem.
- **The Readiness Index Vector:** A data-driven profiling system that maps student performance metrics to show professional skill maturity.
- **The Neural Tutor Interface:** An AI-powered instructional module providing contextual explanations and targeted challenge generation.
- **The Pedagogical Stream:** An automated distribution engine allowing faculty members to create course containers and share resource notes effortlessly.
- **Teacher Analytics Dashboard:** Aggregated visual charts and engagement metrics displaying individual student weaknesses, performance velocities, and class completion statistics.

- **Real-Time Assessment Flow:** Custom assignment creation tools equipped with automated submission recording and grading dashboards.
- **Adaptive Content Delivery:** A backend scheduling layer that suggests practice pathways based on recorded student performance.
- **Global System Monitor:** System monitoring interfaces built for Admins and Super Admins to track platform load, institution logs, and database metrics.

5. Results

5.1. Efficiency

The deployment of DevMerge has drastically optimized the administrative and learning workflow within the academic pipeline. By replacing paper files and third-party code tools with an embedded development and management hub, the system minimizes grading friction and submission loss. Faculty members can deploy laboratory tests instantly, and students receive immediate technical validation, reducing typical feedback loops from several days to zero.

5.2. Data Management

The application offers an optimized MongoDB document cluster that stores all student activity logs, curricula, submissions, and code analytics. Centralizing these data streams eliminates asset tracking redundancy, prevents evaluation mismatch, and ensures high integrity across administrative lookups. Real-time synchronizations allow faculty and HODs to view updated student performance profiles without manual reporting delays.

5.3. Security

DevMerge provides robust protection for user records and institutional data through rigorous cryptographic safeguards. Secure JWT tokens isolate session traffic, and role-based route verification logic stops unauthorized API requests. This ensures that grading databases, institutional metrics, and personal performance profiles remain accessible exclusively to valid stakeholders.

6. Future Scope

While DevMerge introduces a powerful foundation for digital academic management, multiple development tracks can be explored to expand its long-term potential:

- **Telepresence & Peer Streaming:** Integration of low-latency WebRTC modules to support remote lab instructions and code-along sessions directly within the playground.
- **Advanced AI Code Explanation:** Upgrading the Neural Tutor module to leverage fine-tuned local models for deep, line-by-line runtime analysis and refactoring recommendations.
- **Decentralized Sandboxing:** Migrating compilation routines to isolated WebAssembly (Wasm) environments on the client-side to minimize main server computational costs.
- **Automated Anti-Cheating & Plagiarism Pipelines:** Implementing structural similarity algorithms to automatically flag duplicate code logic or copy-pasted patterns.

7. Conclusion

The **DevMerge – An Intelligent Collaborative Learning, Practice and Academic Management Platform** successfully demonstrates how modern web architectures can streamline traditional technical and administrative education. The system replaces fragmented learning tools with a centralized, secure, and data-driven platform.

Through the implementation of a full-stack JavaScript architecture using React 18, Node.js, Express, and MongoDB, the project delivers high runtime performance, multi-role security, and cross-platform responsive scalability. Core functionalities—such as the Algorithm Vault, the Neural Tutor interface, and the Teacher Analytics Dashboard—significantly reduce administrative workloads while sharpening student learning outcomes.

This project is the result of focused design and collaborative implementation by **Navya Aggarwal** under the guidance of **Dr. Shikha Tuteja** at **Chitkara University**. Structured development practices played a vital role in executing the system's intricate component states and route configurations. In conclusion, DevMerge provides a practical, production-ready solution for modern institutions, serving as an intelligent blueprint for the future of technical education software.

8. References

- React 18 Documentation. Available at: <https://react.dev>
- Node.js Documentation. Available at: <https://nodejs.org>
- Express.js Documentation. Available at: <https://expressjs.com>
- MongoDB Atlas & Architecture Manuals. Available at: <https://www.mongodb.com/docs>
- Tailwind CSS Utility Guide. Available at: <https://tailwindcss.com/docs>
- Framer Motion API Reference. Available at: <https://www.framer.com/motion/>
- Zustand State Management Docs. Available at: <https://zustand-demo.pmnd.rs/>
- JSON Web Tokens (JWT) Core Specifications. Available at: <https://jwt.io/introduction>
- Vercel Deployment and Optimization Frameworks. Available at: <https://vercel.com/docs>
- GitHub Version Control and Workflow Actions. Available at: <https://docs.github.com>

9. Appendices

Appendix A: Login Screenshots

Figure A.1: Landing Page



Figure A.2: Login Page

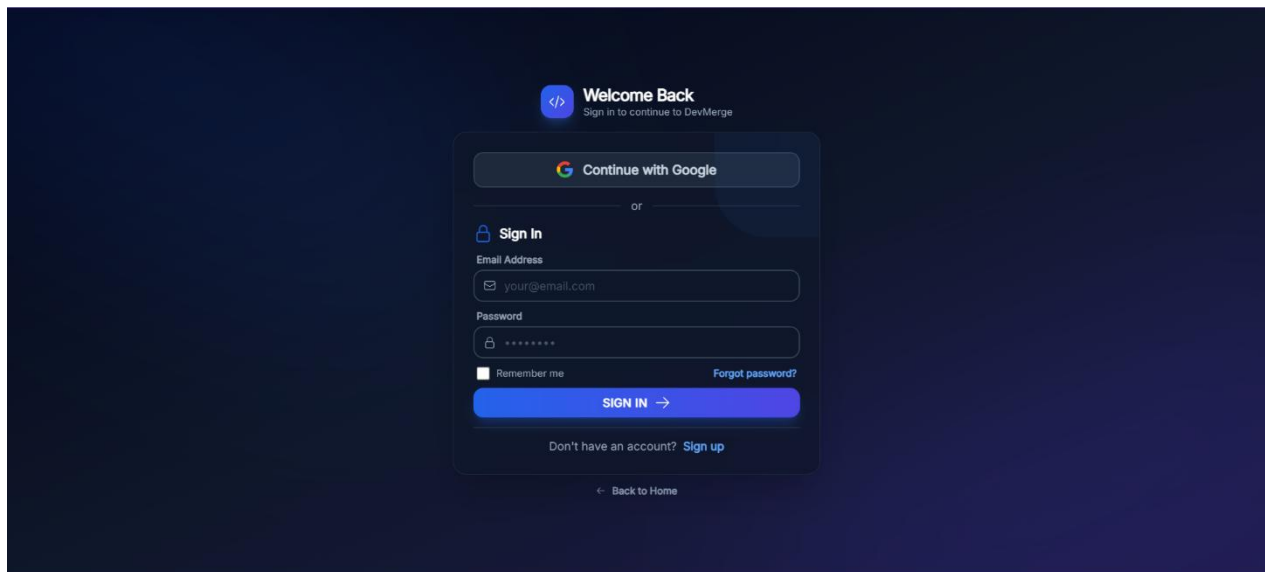


Figure A.3: Student Dashboard

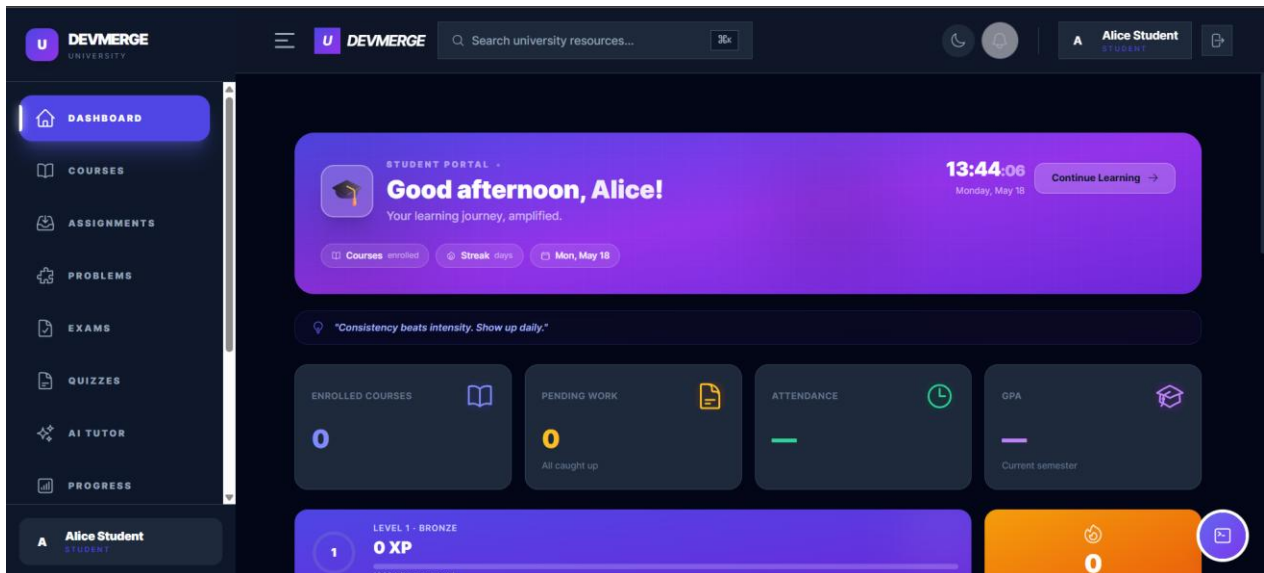


Figure A.4: HOD Dashboard

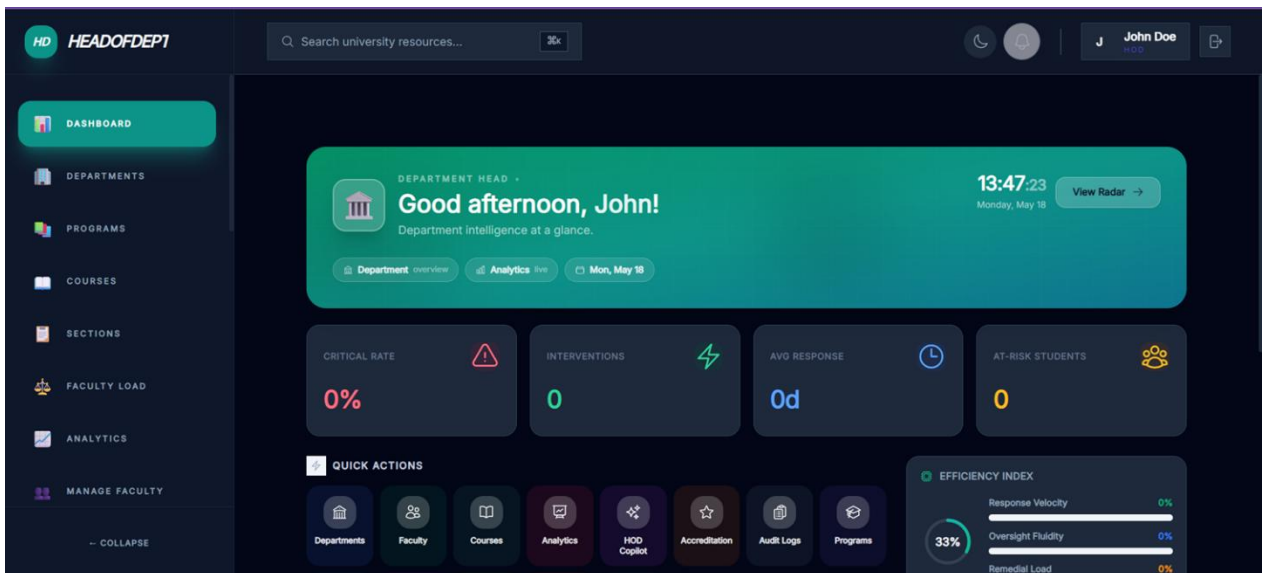


Figure A.5: Faculty Dashboard

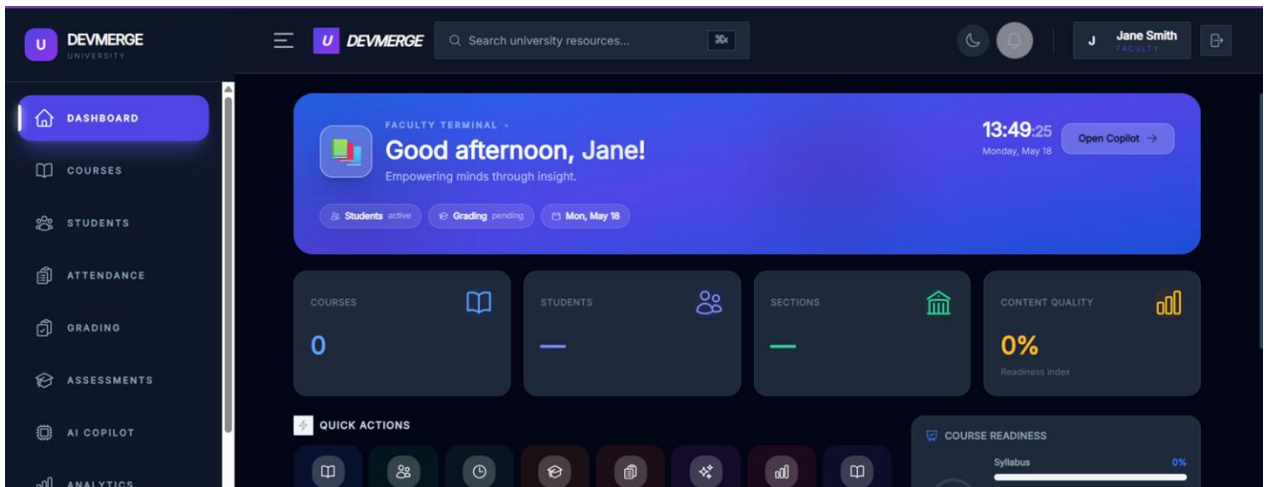


Figure A.6: Code Editor

